Rust

Memory safety enforced through ownership

THE MENTAL MODEL

A library book has one owner; temporary readers follow clear rules.

VALUE → OWNER → BORROW → CHECK

READ THE CODE

```
fn main() {
    let values = [10, 20, 30];
    let sum: i32 = values.iter().sum();
    println!("{sum}");
}
```

FOUR CLUES TO READ IT

- fn declares a function.
- let binds a name to a value.
- iter().sum() traverses and adds values.
- println! is a macro for printing.

WHAT MAKES IT DIFFERENT

Ownership and borrowing restrict unsafe memory patterns at compile time.

No ordinary garbage collector is required for this memory model.

WHERE IT FITS

Systems components, secure services, high-performance tools.

WATCH OUT

The compiler can reject code that works in other languages until ownership is clarified.

TRY ALONE • 60 SEC

Why might two writers to the same mutable value be dangerous?